

TECHNICAL ASSIGNMENT PROGRAMMER

Take Home Test

A. Informasi Umum

Posisi : Programmer

Metode Tes : Take Home Test

Estimasi Waktu Pengerjaan : ± 3 Hari Kerja

B. Studi Kasus

Sebuah klinik pratama masih melakukan sebagian besar proses pelayanan pasien secara manual sehingga sering terjadi antrean yang tidak teratur, pencatatan data pasien yang tersebar, serta kesulitan dalam melihat riwayat pemeriksaan pasien.

Untuk meningkatkan kualitas pelayanan, klinik berencana membangun **Mini Clinic Information System**, yaitu aplikasi berbasis web yang digunakan untuk membantu proses administrasi dan pelayanan pasien secara terintegrasi.

Sebagai **Programmer**, Anda diminta membangun aplikasi yang mencakup proses utama pelayanan klinik, mulai dari pengelolaan data pasien, pendaftaran kunjungan, pengelolaan antrean, hingga pencatatan hasil pemeriksaan dokter.

Aplikasi tidak harus mencakup seluruh proses bisnis klinik, namun harus menunjukkan kemampuan Anda dalam merancang database, membangun REST API, mengembangkan antarmuka pengguna, serta mengintegrasikan frontend dan backend menjadi sebuah aplikasi yang berjalan dengan baik.

C. Teknologi yang Digunakan

Peserta wajib menggunakan teknologi sebagai berikut:

Komponen	Teknologi
Frontend	React.js
Backend	Node.js (Express.js)
Database	PostgreSQL atau MySQL
Authentication	JSON Web Token (JWT)
Version Control	Git

D. Ruang Lingkup Pengerjaan

Peserta diminta membangun aplikasi yang memiliki fitur-fitur sebagai berikut.

1. Authentication

Bangun mekanisme autentikasi pengguna menggunakan **JWT Authentication**.

Role minimal yang harus tersedia:

- Administrator
- Dokter
- Petugas Pendaftaran

Fitur yang harus tersedia:

- Login
- Logout
- Authorization berdasarkan Role

2. Master Data Pasien

Bangun modul pengelolaan data pasien.

Data Pasien

- Nomor Rekam Medis (Auto Generate)
- NIK
- Nama Pasien
- Jenis Kelamin
- Tanggal Lahir
- Nomor Telepon
- Alamat

Fitur

- Tambah Data
- Ubah Data
- Hapus Data
- Detail Data
- Pencarian
- Pagination

Validasi

- NIK tidak boleh duplikat
- Nomor Rekam Medis dibuat secara otomatis

3. Modul Pendaftaran Pasien

Bangun modul pendaftaran pasien.

Data yang dikelola minimal meliputi:

- Pasien
- Dokter
- Poli
- Tanggal Kunjungan
- Jenis Pembayaran

- Keluhan Awal

Status kunjungan:

- Menunggu
- Check In
- Pemeriksaan
- Selesai

4. Modul Antrean

Bangun modul antrean pasien.

Fitur minimal:

- Generate nomor antrean otomatis
- Menampilkan daftar antrean
- Memanggil antrean berikutnya
- Mengubah status antrean

Contoh nomor antrean:

A001

A002

A003

5. Modul Pemeriksaan Dokter

Bangun modul pemeriksaan pasien menggunakan metode **SOAP**.

Subjective

- Keluhan Pasien

Objective

- Tekanan Darah
- Suhu Tubuh
- Berat Badan
- Tinggi Badan

Assessment

- Diagnosa

Plan

- Rencana Terapi

Selain itu, peserta diminta membuat fitur:

- Input Tindakan Medis
- Input Resep Obat
- Riwayat Pemeriksaan Pasien

6. Dashboard

Bangun dashboard sederhana yang menampilkan informasi berikut.

- Total Pasien
- Total Pasien Hari Ini

- Total Antrean Hari Ini
- Total Pasien Menunggu
- Total Pasien Selesai Dilayani

E. REST API Minimum

Backend minimal menyediakan endpoint berikut.

Authentication

POST /login

POST /logout

Patient

GET /patients

GET /patients/{id}

POST /patients

PUT /patients/{id}

DELETE /patients/{id}

Registration

GET /registrations

POST /registrations

PUT /registrations/{id}

Queue

GET /queues

POST /queues

PUT /queues/{id}/call

PUT /queues/{id}/status

Medical Record

POST /medical-records

GET /medical-records/{patientId}

Prescription

POST /prescriptions

GET /prescriptions/{id}

Standar Response API

Seluruh endpoint diharapkan menggunakan format response yang konsisten.

Success Response

JSON

```
{
  "success": true,
  "message": "Success",
  "data": {}
}
```

Error Response

JSON

```
{
  "success": false,
  "message": "Validation Error",
  "errors": {}
}
```

F. Ketentuan Pengerjaan

Peserta diharapkan memperhatikan hal-hal berikut.

- Menggunakan arsitektur aplikasi yang terstruktur dan mudah dikembangkan.
- Mengimplementasikan REST API dengan baik.
- Menggunakan relasi database yang sesuai.
- Menerapkan validasi data pada sisi frontend maupun backend.
- Menangani error (error handling) dengan baik.
- Menggunakan Git selama proses pengembangan.

G. Deliverables

Peserta wajib mengumpulkan:

1. Source Code Frontend (React.js)
2. Source Code Backend (Node.js)
3. File Database (.sql)
4. Entity Relationship Diagram (ERD)
5. Postman Collection
6. File **README.md** yang berisi:
 - Cara instalasi aplikasi
 - Cara menjalankan aplikasi
 - Struktur project
 - Akun login
 - Konfigurasi file .env
 - Cara melakukan migrasi database (jika menggunakan migration)

Ketentuan:

- Wajib menyertakan file .env.example.
 - Konfigurasi database, JWT Secret, dan konfigurasi sensitif lainnya tidak boleh di-hardcode ke dalam source code maupun repository.
7. File .env.example
 8. Repository GitHub/GitLab

Version Control:

- Source code wajib menggunakan Git.
 - Repository harus memiliki riwayat commit (commit history) yang menunjukkan proses pengembangan aplikasi.
 - Hindari hanya melakukan satu commit di akhir pengerjaan.
9. Video demonstrasi aplikasi dengan durasi maksimal 10 menit.

H. Kriteria Penilaian

Aspek Penilaian	Bobot
Database Design (ERD & Relasi Database)	15%
REST API & Backend Implementation	20%
Frontend Implementation (React.js)	20%
Authentication & Authorization	10%
Clean Code & Struktur Project	10%
Validasi Data & Error Handling	10%
Dokumentasi (README, Postman, ERD)	10%
Git Commit History	5%
Total	100%

Catatan

- Peserta diperbolehkan menggunakan library atau package pendukung selama tidak mengubah teknologi utama yang telah ditentukan.
- Peserta diharapkan mengerjakan tes secara mandiri.
- Tampilan antarmuka (UI/UX) tidak harus sama dengan aplikasi tertentu, namun diharapkan memiliki desain yang rapi, mudah digunakan, dan konsisten.
- Apabila terdapat asumsi atau penyederhanaan proses bisnis, peserta dapat menjelaskannya pada dokumen **README.md**.